INTRODUCCIÓN A ANSIBLE

Ansible es una plataforma de automatización de TI que permite implementar y gestionar sistemas, orquestar servicios y automatizar tareas administrativas sin que sea necesario escribir scripts complejos para cada acción. Opera mediante la ejecución de scripts de automatización escritos en YAML, conocidos como *playbooks*, que definen un conjunto de tareas a realizar en los nodos. La ejecución de estas tareas se realiza de forma **idempotente** y **declarativa**, es decir, ejecutar una tarea múltiples veces sobre el mismo sistema produce siempre el mismo resultado sin realizar cambios innecesarios.

Uno de los rasgos más importantes es su funcionamiento sin agentes, lo que significa que Ansible se comunica con los servidores a través de protocolos estándar, como SSH, sin requerir software especial en cada máquina. Esto presenta las siguientes ventajas:

- No es necesario mantener versiones en cada sistema.
- Mejora la seguridad, pues no introduce componentes adicionales que puedan incluir vulnerabilidades.
- Simplifica la arquitectura y facilita la escalabilidad y gestión de entornos heterogéneos.

Tipos de nodos:

- Nodos de control: son los servidores o equipos desde donde se ejecutan los playbooks y se dirigen las tareas de automatización. En los nodos de control se encuentra el Inventario, que es una lista de los nodos que son gestionados.
- Nodos gestionados: son los sistemas destino donde se aplican las configuraciones y se ejecutan comandos.

CONCEPTOS BÁSICOS

Module (Módulo)

Encapsula la lógica necesaria para realizar una acción específica y concreta. Viene predefinido, evitando la necesidad de escribir código personalizado.
Ejemplo: *ansible.builtin.apt* (sirve para administrar paquetes de software mediante el gestor de paquetes apt en distribuciones Linux basadas en Debian).

Task (Tarea)

Es la unidad individual de ejecución dentro de un *play* en Ansible. Representa un paso concreto e indivisible necesario para llevar a un nodo gestionado a un estado específico. Cada tarea invoca un único módulo de Ansible y le proporciona los parámetros requeridos para su ejecución.
Ejemplo:

```
- name: Instalar el paquete Curl
  ansible.builtin.apt:
    name: curl
    state: present
```

Play
Mapeo explícito entre una o varias tareas y uno o varios nodos gestionados para llevar a cabo una configuración.
Ejemplo:

```
- name: Configurar los servidores de base de datos
  hosts: db_servers
  become: true
  tasks:
    - name: Instalar el paquete PostgreSQL
      ansible.builtin.package:
        name: postgresql-server
        state: present
    - name: Iniciar servicio PostgreSQL
      ansible.builtin.service:
        name: postgresql
        state: started
```

Playbook
Fichero en formato YAML que contiene uno o varios *plays*. Es el script principal de orquestación donde se define el estado deseado de toda la infraestructura.

Role (Rol)
Permite empaquetar todo lo necesario para realizar una configuración específica en una estructura de carpetas estandarizadas, facilitando su reutilización. Para usarlo, se invoca desde un *play*.
Estructura de carpetas:

```
nombre_del_rol/
├── defaults/ → define las variables por defecto del rol. Tienen la prioridad más baja.
│   └── main.yml
├── vars/ → define variables internas del rol. Tienen una prioridad muy alta.
│   └── main.yml
├── tasks/ → lista principal de tareas que ejecuta el rol.
│   └── main.yml
├── handlers/ → disparadores. Tareas que se ejecutan cuando son notificadas por otras.
│   └── main.yml
├── templates/ → archivos de plantilla con extensión .j2 (Jinja2).
├── files/ → archivos estáticos (scripts, binarios, certificados, imágenes, etc.).
├── meta/ → metadatos del rol, como el autor, la licencia, las plataformas soportadas, etc.
│   └── main.yml
└── README.md
```

* Para generar automáticamente la estructura básica de un rol:

```
ansible-galaxy role init nombre_del_rol
```

GESTIÓN DE INVENTARIOS

Para una lista pequeña de equipos (nodos) se puede usar un archivo .ini, con la siguiente forma:
[myhosts] → nombre del grupo
192.0.2.50
192.0.2.51
192.0.2.52

Cuando la cantidad de nodos gestionados aumenta, es recomendable utilizar archivos .yml para definir el inventario. Esto permite asignar nombres y variables a los hosts y crear grupos y metagrupos (grupo de grupos).

```
leafs: → grupo 1
  hosts:
    leaf01: → nombre del host 1
      ansible_host: 192.0.2.100
    leaf02: → nombre del host 2
      ansible_host: 192.0.2.110

spines: → grupo 2
  hosts:
    spine01: → nombre del host 1
      ansible_host: 192.0.2.120
    spine02: → nombre del host 2
      ansible_host: 192.0.2.130

network: → metagrupo 1
  children:
    leafs: → se añaden los hosts del grupo 1
    spines: → se añaden los hosts del grupo 2

datacenter: → metagrupo 2
  children:
    network: → se añaden los hosts del metagrupo 1
```

* Verificar inventario:

```
ansible-inventory -i inventory.ini --list
```

HERRAMIENTA CLI

La herramienta de línea de comandos principal es *ansible-playbook*. Su finalidad es ejecutar un playbook sobre los nodos gestionados que corresponda.
Sintaxis básica:

```
ansible-playbook ruta/fichero_playbook.yml [opciones]
```

Opciones comunes:

Opción corta	Opción larga	Descripción
`-i`	`--inventory`	Especificar la ruta del inventario o una lista de hosts separados por comas.
`-l`	`--limit`	Restringir la ejecución a hosts o grupos específicos del inventario.
`-e`	`--extra-vars`	Definir variables adicionales en tiempo de ejecución (`clave=valor` o `@fichero.yml`).
`-t`	`--tags`	Ejecutar únicamente las tareas y los *plays* asociados a las etiquetas especificadas.
—	`--skip-tags`	Omitir las tareas y los *plays* asociados a las etiquetas especificadas.
`-b`	`--become`	Activar la elevación de privilegios para ejecutar tareas como otro usuario (por defecto, `root`).
`-K`	`--ask-become-pass`	Solicitar por consola la contraseña para la elevación de privilegios.
`-k`	`--ask-pass`	Solicitar por consola la contraseña de conexión al host remoto.
`-u`	`--user`	Especificar el usuario remoto con el que se establece la conexión.
`-C`	`--check`	Activar el modo de comprobación (*dry-run*): intentar predecir los cambios sin aplicarlos.
`-D`	`--diff`	Mostrar las diferencias de contenido en los archivos y plantillas modificados. Especialmente útil junto con `--check`.

`-v`	`--verbose`	Incrementar el nivel de detalle de la salida. Se pueden acumular (`-vvv`, `-vvvv`, etc.) para facilitar la depuración.
—	`--syntax-check`	Validar la sintaxis del playbook sin ejecutar sus tareas.
—	`--list-tasks`	Listar las tareas que se ejecutarían en el playbook sin ejecutarlas.
—	`--list-hosts`	Listar los hosts que coinciden con los patrones definidos en los *plays*.
—	`--list-tags`	Listar las etiquetas (*tags*) disponibles en las tareas.
—	`--step`	Solicitar confirmación interactiva antes de ejecutar cada tarea.
—	`--start-at-task`	Iniciar la ejecución del playbook a partir de una tarea específica por su nombre.
`-f`	`--forks`	Definir el número de procesos paralelos que se utilizarán (por defecto: 5).
`-T`	`--timeout`	Configurar el tiempo máximo de espera de conexión en segundos.
—	`--flush-cache`	Limpiar la caché de *facts* de todos los hosts del inventario.
`-J`	`--ask-vault-pass`	Solicitar por consola la contraseña para descifrar los datos protegidos con Ansible Vault.
—	`--vault-password-file`	Especificar la ruta al fichero que contiene la contraseña de Ansible Vault.